AI ASSISTED CODING ASSIGNMENT WEEK-13.1

Name:SiriChandana
HT No:2303A52004
Batch:31

Question:1

Task Description #1 (Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
• Instructions:
o Prompt AI to identify duplicate logic and replace it with functions or classes.
o Ensure the refactored code maintains the same output.
o Add docstrings to all functions.
• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

• Expected Output:
o Refactored code with a reusable function and no duplication.
o Well documented code

Code:

```python
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        tuple: A tuple containing (area, perimeter).
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter


def print_rectangle_info(length, width):
    """
    Print the area and perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.
    """
    area, perimeter = calculate_rectangle_properties(length, width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")


# Use the reusable function
print_rectangle_info(5, 10)
print_rectangle_info(7, 12)
print_rectangle_info(10, 15)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\1.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Justification:

- The repeated rectangle calculations were moved into a function, which avoids writing the same code multiple times.
- This makes the program cleaner, reusable, and easier to maintain if the logic needs to be changed later.

Question:2

Task Description #2 (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

Week7
-
Monda
y

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

• Expected Output:

o Code with a function calculate_total(price) that can be reused for multiple price inputs.

o Well documented code

Code:

```python
def calculate_tax(price, tax_rate=0.18):
    """
    Calculate the tax amount for a given price.
    Args:
        price (float): The base price before tax.
        tax_rate (float): The tax rate as a decimal (default: 0.18 for 18%).
    Returns:
        float: The calculated tax amount.
    """
    return price * tax_rate
def calculate_total(price, tax_rate=0.18):
    """
    Calculate the total price including tax.
    Args:
        price (float): The base price before tax.
        tax_rate (float): The tax rate as a decimal (default: 0.18 for 18%).
    Returns:
        float: The total price after adding tax.
    """
    tax = calculate_tax(price, tax_rate)
    return price + tax
def display_price_summary(price, tax_rate=0.18):
    """
    Display a formatted summary of price, tax, and total.
    Args:
        price (float): The base price before tax.
        tax_rate (float): The tax rate as a decimal (default: 0.18 for 18%).
    """
    total = calculate_total(price, tax_rate)
    tax = calculate_tax(price, tax_rate)
    print(f"Price: ${price:.2f}")
    print(f"Tax: ${tax:.2f}")
    print(f"Total Price: ${total:.2f}\n")
# Main execution
if __name__ == "__main__":
    display_price_summary(250)
    display_price_summary(500)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\2.py"
Price: $250.00
Tax: $45.00
Total Price: $295.00

Price: $500.00
Tax: $90.00
Total Price: $590.00
```

Justification:

- The tax and total price calculations were placed inside a reusable function `calculate_total(price)`.
- This improves modularity and readability, allowing the same function to be used for different price values

Question:3

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:

o Accept marks as a parameter.

o Return the corresponding grade as a string.

o The grading logic must strictly follow the conditions below:

- Marks ≥ 90 and ≤ 100 → "Grade A"
- Marks ≥ 80 → "Grade B"
- Marks ≥ 70 → "Grade C"
- Marks ≥ 40 → "Grade D"
- Marks ≥ 0 → "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:

o The class

o The method (with parameter and return descriptions)

5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
marks = 72
if marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

• Define a class named GradeCalculator.

• Implement a method calculate_grade(self, marks) inside the class.
• Create an object of the class.
• Call the method for different student marks.
• Print the returned grade values.

Code:

```python
class GradeCalculator:
    """
    A class to calculate and assign grades based on student marks.
    This class encapsulates the grading logic, eliminating redundant
    conditional statements and providing a reusable interface for
    grade calculation.
    """
    def calculate_grade(self, marks):
        """
        Calculate the grade based on the provided marks.
        Parameters:
        marks (int): The marks obtained by the student (0-100).
        Returns:
        str: The corresponding grade as a string.
        """
        if marks >= 90 and marks <= 100:
            return "Grade A"
        elif marks >= 80:
            return "Grade B"
        elif marks >= 70:
            return "Grade C"
        elif marks >= 40:
            return "Grade D"
        else:
            return "Fail"
# Create an instance of the GradeCalculator class
calculator = GradeCalculator()
# Call the method for different student marks
marks1 = 85
grade1 = calculator.calculate_grade(marks1)
print(f"Marks: {marks1} -> {grade1}")
marks2 = 72
grade2 = calculator.calculate_grade(marks2)
print(f"Marks: {marks2} -> {grade2}")
marks3 = 95
grade3 = calculator.calculate_grade(marks3)
print(f"Marks: {marks3} -> {grade3}")
marks4 = 35
grade4 = calculator.calculate_grade(marks4)
print(f"Marks: {marks4} -> {grade4}")
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\3.py"
Marks: 85 -> Grade B
Marks: 72 -> Grade C
Marks: 95 -> Grade A
Marks: 35 -> Fail
```

Justification:

- The grading logic was moved into a class `GradeCalculator`, which organizes the code using object-oriented principles.
- The method `calculate_grade()` can be reused for multiple students without rewriting conditional statements

Question:4

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

num = int(input("Enter number: "))

square = num * num

print("Square:", square)

• Expected Output:

o Modular code using functions like get_input(), calculate_square(), and display_result().

Code:

```
def get_input():
    """Get a number from the user."""
    num = int(input("Enter number: "))
    return num


def calculate_square(num):
    """Calculate the square of a number."""
    square = num * num
    return square


def display_result(square):
    """Display the result."""
    print("Square:", square)


def main():
    """Main function to orchestrate the program flow."""
    num = get_input()
    square = calculate_square(num)
    display_result(square)


if __name__ == "__main__":
    main()
```

Output:

```
Enter number: 11
Square: 121
```

Justification:

- The program was divided into separate functions like input, processing, and output, improving code structure.
- This makes the program more readable and easier to debug or modify.

Question:5

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Code:

```python
class EmployeeSalaryCalculator:
    def __init__(self, salary, tax_rate=0.2):
        self.salary = salary
        self.tax_rate = tax_rate
    def calculate_tax(self):
        return self.salary * self.tax_rate
    def calculate_net_salary(self):
        return self.salary - self.calculate_tax()
    def display_salary_info(self):
        tax = self.calculate_tax()
        net = self.calculate_net_salary()
        print(f"Gross Salary: ${self.salary:,.2f}")
        print(f"Tax ({self.tax_rate*100}%): ${tax:,.2f}")
        print(f"Net Salary: ${net:,.2f}")
# Usage
calculator = EmployeeSalaryCalculator(50000)
calculator.display_salary_info()
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\5.py"
Gross Salary: $50,000.00
Tax (20.0%): $10,000.00
Net Salary: $40,000.00
```

Justification:

- The salary and tax calculation logic was implemented inside a class, which follows object-oriented programming principles.
- Encapsulation helps organize related data and methods together, making the code scalable.

Question:6

Task 6 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity
o Data structure choice
Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification
Code:

```
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
print("Access Granted" if name in users else "Access Denied")
```

Output:

```
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\6.py"
Enter username: user
Access Denied
```

Justification:

- The linear search was replaced with a set/dictionary lookup, which provides faster search operations.
- This improves time complexity from O(n) to O(1) for checking whether a user exists.

Question:7
Task 7 – Refactoring the Library Management System
Problem Statement
You are provided with a poorly structured Library Management script that:
• Contains repeated conditional logic
• Does not use reusable functions
• Lacks documentation
• Uses print-based procedural execution
• Does not follow modular programming principles
Your task is to refactor the code into a proper format
1. Create a module library.py with functions:
o add_book(title, author, isbn)
o remove_book(isbn)
o search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.
Given Code
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book

```
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
library_db[isbn] = {"title": title, "author": author}
print("Book added successfully.")
else:
print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
del library_db[isbn]
print("Book removed successfully.")
else:
print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
print("Book Found:", library_db[isbn])
else:
print("Book not found.")
```

Code:

```python
"""
Library Management System
A modular library management application with proper documentation.
"""
library_db = {}
def add_book(title, author, isbn):
    """
    Add a new book to the library database.
    Args:
        title (str): The title of the book.
        author (str): The author of the book.
        isbn (str): The ISBN identifier for the book.
    Returns:
        bool: True if book was added successfully, False if book already exists.
    Example:
        >>> add_book("Python Basics", "John Doe", "101")
        True
    """
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        print("✓ Book added successfully.")
        return True
    else:
        print("✗ Book already exists.")
        return False
def remove_book(isbn):
    """
    Remove a book from the library database by ISBN.
    Args:
        isbn (str): The ISBN identifier of the book to remove.
    Returns:
        bool: True if book was removed successfully, False if book not found.
    Example:
        >>> remove_book("101")
        True
    """
    if isbn in library_db:
        del library_db[isbn]
        print("✓ Book removed successfully.")
        return True
    else:
        print("✗ Book not found.")
        return False
def search_book(isbn):
    """
    Search for a book in the library database by ISBN.
    Args:
        isbn (str): The ISBN identifier of the book to search.
    Returns:
        dict: Book details if found, None if book not found.
    Example:
        >>> search_book("101")
        {'title': 'Python Basics', 'author': 'John Doe'}
    """
    if isbn in library_db:
        print(f"✓ Book Found: {library_db[isbn]}")
        return library_db[isbn]
    else:
        print("✗ Book not found.")
        return None
# Main execution
if __name__ == "__main__":
    add_book("Python Basics", "John Doe", "101")
    add_book("AI Fundamentals", "Jane Smith", "102")
    search_book("101")
    remove_book("101")
    search_book("101")
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\7.py"
✓ Book added successfully.
✓ Book added successfully.
✓ Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
✓ Book removed successfully.
✗ Book not found.
```

Justification:

- The code was organized into separate reusable functions in a module, improving modularity.
- Adding docstrings and documentation makes the program easier to understand and maintain.

Question:8

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

• Global variables.

• Inefficient loop.

• No functions or modularity.

Task for Students:

• Refactor into a clean reusable function (generate_fibonacci).

• Add docstrings and test cases.

• Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
c=a+b
print(c)
```

Code:

```python
def generate_fibonacci(n):
    """
    Generate Fibonacci series up to n terms.

    Args:
        n (int): Number of Fibonacci terms to generate
    Returns:
        list: List containing the first n Fibonacci numbers
    Raises:
        ValueError: If n is negative

    Examples:
        >>> generate_fibonacci(5)
        [0, 1, 1, 2, 3]
        >>> generate_fibonacci(1)
        [0]
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")
    if n == 0:
        return []
    fibonacci_series = []
    a, b = 0, 1
    for _ in range(n):
        fibonacci_series.append(a)
        a, b = b, a + b
    return fibonacci_series
def print_fibonacci(n):
    """Print Fibonacci series up to n terms."""
    if n <= 0:
        print("Please enter a positive integer")
        return
    series = generate_fibonacci(n)
    print(f"Fibonacci series ({n} terms): {series}")
# Test cases
if __name__ == "__main__":
    # Test 1: Normal case
    assert generate_fibonacci(5) == [0, 1, 1, 2, 3]
    # Test 2: Edge case - zero terms
    assert generate_fibonacci(0) == []
    # Test 3: Single term
    assert generate_fibonacci(1) == [0]
    # Test 4: Larger series
    assert generate_fibonacci(8) == [0, 1, 1, 2, 3, 5, 8, 13]
    print("✓ All tests passed!")
    # Interactive usage
    n = int(input("Enter limit: "))
    print_fibonacci(n)
```

Output:

```
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\8.py"
✓ All tests passed!
Enter limit: 5
Fibonacci series (5 terms): [0, 1, 1, 2, 3]
```

Justification:

- The program was refactored into a reusable function `generate_fibonacci()`, which improves modularity and avoids global variables.

- Using functions and docstrings makes the code cleaner, easier to test, and reusable for different input values.

Question:9

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

• Inefficient prime checking.

• No functions.

• Hardcoded inputs.

Task for Students:

• Refactor into is_prime(n) and is_twin_prime(p1, p2).

• Add docstrings and optimize.

• Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
if a%i==0:
fa=1
fb=0
for i in range(2,b):
if b%i==0:
fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")
```

Code:

```python
def is_prime(n):
    """
    Check if a number is prime.
    Args:
        n: Integer to check for primality
    Returns:
        Boolean: True if prime, False otherwise
    """
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True
def is_twin_prime(p1, p2):
    """
    Check if two numbers form a twin prime pair.
    Args:
        p1: First number
        p2: Second number
    Returns:
        Boolean: True if both are prime and differ by 2
    """
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
def find_twin_primes(start, end):
    """
    Generate all twin prime pairs in a given range.
    Args:
        start: Range start (inclusive)
        end: Range end (inclusive)
    Returns:
        List of tuples containing twin prime pairs
    """
    twin_primes = []
    for num in range(start, end - 1):
        if is_twin_prime(num, num + 2):
            twin_primes.append((num, num + 2))
    return twin_primes
# Main execution
if __name__ == "__main__":
    a, b = 11, 13
    if is_twin_prime(a, b):
        print(f"{a} and {b} are Twin Primes")
    else:
        print(f"{a} and {b} are Not Twin Primes")
    # Find all twin primes in range
    twin_primes_list = find_twin_primes(10, 50)
    print(f"\nTwin primes between 10 and 50: {twin_primes_list}")
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\9.py"
11 and 13 are Twin Primes

Twin primes between 10 and 50: [(11, 13), (17, 19), (29, 31), (41, 43)]
```

Justification:

- Separate functions `is_prime()` and `is_twin_prime()` were created to avoid repeated prime-checking logic.
- This improves code readability, reusability, and efficiency compared to the original hard-coded implementation.

Question:10

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

```
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
print("Monkey")
elif year % 12 == 1:
print("Rooster")
elif year % 12 == 2:
print("Dog")
elif year % 12 == 3:
print("Pig")
elif year % 12 == 4:
print("Rat")
elif year % 12 == 5:
```

```
print("Ox")
elif year % 12 == 6:
print("Tiger")
elif year % 12 == 7:
print("Rabbit")
elif year % 12 == 8:
print("Dragon")
elif year % 12 == 9:
print("Snake")
elif year % 12 == 10:
print("Horse")
elif year % 12 == 11:
print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

Code:

```python
"""
Chinese Zodiac Finder
A clean, modular implementation to determine the Chinese Zodiac sign for a given year.
"""
def get_zodiac(year):
    """
    Determine the Chinese Zodiac sign for a given year.
    Args:
        year (int): The year to look up.
    Returns:
        str: The corresponding Chinese Zodiac sign.
    """
    zodiac_signs = [
        "Monkey",
        "Rooster",
        "Dog",
        "Pig",
        "Rat",
        "Ox",
        "Tiger",
        "Rabbit",
        "Dragon",
        "Snake",
        "Horse",
        "Goat"
    ]
    return zodiac_signs[year % 12]
def main():
    """Main function to handle user input and display the Chinese Zodiac sign."""
    try:
        year = int(input("Enter a year: "))
        zodiac = get_zodiac(year)
        print(f"The Chinese Zodiac sign for {year} is: {zodiac}")
    except ValueError:
        print("Error: Please enter a valid integer year.")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\10.py"
Enter a year: 2006
The Chinese Zodiac sign for 2006 is: Dog
```

Justification:

- The long if-elif chain was replaced with a list/dictionary, making the program shorter and easier to maintain.
- The logic was moved into a reusable function `get_zodiac(year)`, improving modularity and readability.

Question:11

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

• 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 □ (Harshad Number)

• 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer □ (Not Harshad)

Problem Statement

The current implementation:
• Mixes logic and input handling
• Uses redundant variables
• Does not use reusable functions properly
• Returns print statements instead of boolean values
• Lacks documentation
You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
digit = temp % 10
sum_digits = sum_digits + digit
temp = temp // 10
if sum_digits != 0:
if num % sum_digits == 0:
print("True")
else:
print("False")
else:
print("False")
```

You must:
1. Create a reusable function: is_harshad(number)
2. The function must:
o Accept an integer parameter.
o Return True if the number is divisible by the sum of its digits.
o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

Code:

```python
"""
Harshad (Niven) Number Checker - Refactored Implementation
A Harshad number is a positive integer that is divisible by the sum of its digits.
Examples: 18 (1+8=9, 18÷9=2), 21 (2+1=3, 21÷3=7)
"""
def calculate_digit_sum(number: int) -> int:
    """
    Calculate the sum of digits of an absolute number.
    Args:
        number: An integer value
    Returns:
        The sum of the absolute digits of the number
    """
    return sum(int(digit) for digit in str(abs(number)))
def is_harshad(number: int) -> bool:
    """
    Check if a number is a Harshad (Niven) number.
    A Harshad number is divisible by the sum of its digits.
    Only positive integers can be Harshad numbers.
    Args:
        number: An integer to check
    Returns:
        True if the number is a Harshad number, False otherwise
    """
    if number <= 0:
        return False
    digit_sum = calculate_digit_sum(number)
    return number % digit_sum == 0
def main():
    """Main function to handle user input and display results."""
    try:
        user_input = int(input("Enter a number: "))
        result = is_harshad(user_input)
        print(result)
    except ValueError:
        print("Invalid input. Please enter an integer.")
if __name__ == "__main__":
    main()
```

Output:

```
Enter a number: 1
True
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\11.py"
Enter a number: 55
False
```

Justification:

- A function `is_harshad(number)` was created to separate logic from user input.
- The function now returns True/False instead of printing, making the code reusable and suitable for testing.

Question:12

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

• Calculates the full factorial (inefficient for large n)
• Mixes input handling with business logic
• Uses print statements instead of return values
• Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
fact = fact * i
i = i + 1
count = 0
while fact % 10 == 0:
count = count + 1
fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
o Accept a non-negative integer n.
o Return the number of trailing zeros in n!.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

Code:

```python
def count_trailing_zeros(n):
    """
    Calculate the number of trailing zeros in n! (factorial of n).
    Trailing zeros are produced by factors of 10, which come from pairs of 2 and 5.
    Since there are always more factors of 2 than 5 in n!, we only need to count
    factors of 5.
    Args:
        n: Non-negative integer.
    Returns:
        Number of trailing zeros in n!.
    Raises:
        ValueError: If n is negative.
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")
    zeros = 0
    power_of_5 = 5
    while power_of_5 <= n:
        zeros += n // power_of_5
        power_of_5 *= 5
    return zeros
def main():
    """Main function to handle user interaction."""
    try:
        n = int(input("Enter a number: "))
        result = count_trailing_zeros(n)
        print(f"Trailing zeros: {result}")
    except ValueError as e:
        print(f"Error: {e}")
if __name__ == "__main__":
    main()
```

Output:

```
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\12.py"
Enter a number: 12
Trailing zeros: 2
```

Start

Justification:

- Instead of calculating the full factorial, the optimized method counts multiples of 5, which improves efficiency.
- The logic was placed inside `count_trailing_zeros(n)`, making the program modular and easier to reuse.

Question:13

Task Description #13: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

• Choose suitable data structures from the provided list.

• Provide 2–3 sentence justification per feature.

• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.

• Working Python implementation.

Code:

```python
from collections import deque, defaultdict
import heapq
from typing import Dict, List, Tuple, Optional
"""
Smart Airport Management System - Data Structure Implementation
Demonstrates optimal data structures for airport operations.
"""

# ============================================================================
# 1. PASSENGER CHECK-IN QUEUE (Queue - FIFO)
# ============================================================================
class CheckInQueue:
    """
    Uses Queue (deque) for FIFO processing.
    Justification: Passengers must be processed in arrival order for fairness.
    FIFO ensures no passenger is skipped, and deque provides O(1) append/pop.
    """
    def __init__(self):
        self.queue = deque()
        self.counter = 0

    def add_passenger(self, passenger_id: str, flight: str) -> None:
        """Add passenger to check-in queue."""
        self.counter += 1
        self.queue.append({"passenger_id": passenger_id, "flight": flight, "position": self.counter})

    def process_passenger(self) -> Optional[Dict]:
        """Process next passenger in line."""
        return self.queue.popleft() if self.queue else None

    def queue_length(self) -> int:
        return len(self.queue)
```

```python
# ============================================================================
# 2. EMERGENCY LANDING PRIORITY (Min Heap)
# ============================================================================
class EmergencyLandingQueue:
    """
    Uses Min Heap (priority queue) with priority levels.
    Justification: Emergency aircraft need O(log n) insertion and O(1) peek.
    Priority values (1=critical, 5=normal) ensure emergencies are always first.
    """
    def __init__(self):
        self.heap = []
        self.counter = 0

    def add_aircraft(self, aircraft_id: str, priority: int) -> None:
        """Add aircraft with priority (1=critical, 5=normal)."""
        self.counter += 1
        heapq.heappush(self.heap, (priority, self.counter, aircraft_id))

    def land_aircraft(self) -> Optional[str]:
        """Get highest priority aircraft for landing."""
        if self.heap:
            return heapq.heappop(self.heap)[2]
        return None

    def next_aircraft(self) -> Optional[str]:
        """Peek next aircraft without removing."""
        return self.heap[0][2] if self.heap else None
```

```python
# ============================================================================
# 3. FLIGHT INFORMATION LOOKUP (Hash Table/Dictionary)
# ============================================================================
class FlightDatabase:
    """
    Uses Hash Table (Dictionary) for O(1) lookup by Flight ID.
    Justification: Direct key-value access provides instant retrieval.
    Faster than linear search through lists or arrays.
    """
    def __init__(self):
        self.flights: Dict[str, Dict] = {}

    def add_flight(self, flight_id: str, destination: str, departure_time: str, gate: int) -> None:
        """Add flight information."""
        self.flights[flight_id] = {
            "destination": destination,
            "departure_time": departure_time,
            "gate": gate,
            "status": "Scheduled"
        }

    def get_flight(self, flight_id: str) -> Optional[Dict]:
        """Retrieve flight info by ID in O(1) time."""
        return self.flights.get(flight_id, None)

    def update_status(self, flight_id: str, status: str) -> bool:
        """Update flight status."""
        if flight_id in self.flights:
            self.flights[flight_id]["status"] = status
            return True
        return False
```

```python
# ============================================================================
# 4. AIR ROUTE CONNECTIVITY (Graph - Adjacency List)
# ============================================================================
class AirRouteNetwork:
    """
    Uses Adjacency List (Graph) with defaultdict.
    Justification: Represents airport connections efficiently for route queries.
    O(1) storage per edge and O(degree) traversal; better than adjacency matrix.
    """
    def __init__(self):
        self.graph: Dict[str, List[Tuple[str, int]]] = defaultdict(list)

    def add_route(self, airport1: str, airport2: str, distance: int) -> None:
        """Add bidirectional route between airports."""
        self.graph[airport1].append((airport2, distance))
        self.graph[airport2].append((airport1, distance))

    def get_connected_airports(self, airport: str) -> List[Tuple[str, int]]:
        """Get all directly connected airports."""
        return self.graph.get(airport, [])

    def find_routes_bfs(self, start: str, end: str) -> Optional[List[str]]:
        """Find path between airports using BFS."""
        if start == end:
            return [start]
        visited = set()
        queue = deque([(start, [start])])
        while queue:
            current, path = queue.popleft()
            if current in visited:
                continue
            visited.add(current)
            for neighbor, _ in self.graph[current]:
                if neighbor == end:
                    return path + [neighbor]
                if neighbor not in visited:
                    queue.append((neighbor, path + [neighbor]))
        return None
```

```python
# ============================================================================
# 5. BOARDING PROCESS - OVERHEAD LUGGAGE (Stack - LIFO)
# ============================================================================
class OverheadLuggageManagement:
    """
    Uses Stack (LIFO - Last-In First-Out) for cabin storage.
    Justification: Passengers board sequentially; last luggage stored is first to unload.
    Stack structure matches real-world overhead bin constraints perfectly.
    """
    def __init__(self, capacity: int = 50):
        self.stack = []
        self.capacity = capacity

    def store_luggage(self, passenger_id: str, luggage_id: str) -> bool:
        """Store luggage in overhead bin."""
        if len(self.stack) < self.capacity:
            self.stack.append({"passenger": passenger_id, "luggage": luggage_id})
            return True
        return False

    def retrieve_luggage(self) -> Optional[Dict]:
        """Remove luggage (LIFO order)."""
        return self.stack.pop() if self.stack else None

    def bin_capacity_status(self) -> Tuple[int, int]:
        """Return (used, total) capacity."""
        return (len(self.stack), self.capacity)
```

```python
# ============================================================================
# DEMO USAGE
# ============================================================================
if __name__ == "__main__":
    # 1. Check-in Queue
    checkin = CheckInQueue()
    checkin.add_passenger("P001", "AA100")
    checkin.add_passenger("P002", "UA200")
    print(f"Next passenger: {checkin.process_passenger()}")

    # 2. Emergency Landing
    emergency = EmergencyLandingQueue()
    emergency.add_aircraft("AC001", 3)  # Normal
    emergency.add_aircraft("AC002", 1)  # Critical
    print(f"Priority aircraft: {emergency.land_aircraft()}")

    # 3. Flight Database
    flights = FlightDatabase()
    flights.add_flight("AA100", "NYC", "10:30", 5)
    print(f"Flight info: {flights.get_flight('AA100')}")

    # 4. Route Network
    routes = AirRouteNetwork()
    routes.add_route("LAX", "NYC", 2450)
    routes.add_route("NYC", "JFK", 0)
    print(f"Connected to NYC: {routes.get_connected_airports('NYC')}")

    # 5. Luggage Stack
    luggage = OverheadLuggageManagement(capacity=5)
    luggage.store_luggage("P001", "L001")
    luggage.store_luggage("P002", "L002")
    print(f"Retrieved: {luggage.retrieve_luggage()}")
    print(f"Bin status: {luggage.bin_capacity_status()}")
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\13.py"
Next passenger: {'passenger_id': 'P001', 'flight': 'AA100', 'position': 1}
Priority aircraft: AC002
Flight info: {'destination': 'NYC', 'departure_time': '10:30', 'gate': 5, 'status': 'Scheduled'}
Connected to NYC: [('LAX', 2450), ('JFK', 0)]
Retrieved: {'passenger': 'P002', 'luggage': 'L002'}
Bin status: (1, 5)
```

Justification:

- Appropriate data structures were selected based on real-world system behavior (queue for passengers, priority queue for emergencies).
- Choosing the right data structures improves efficiency, faster access, and better system performance.

Question:14

Task Description #14: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

• Select appropriate data structures from the list.
• Justify your choices clearly.
• Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

• Table mapping features to data structures with justification.
• Working Python code with docstrings and comments.

Code:

```python
from collections import deque, Counter
import heapq
from typing import Dict, List, Set, Tuple
"""
Smart Social Media Platform - Data Structure Selection & Implementation
Task #14: Analyzes feature requirements and implements optimal data structures
"""
# ============================================================================
# DATA STRUCTURE SELECTION TABLE
# ============================================================================
"""
FEATURE                     | DATA STRUCTURE      | JUSTIFICATION
----------------------------|---------------------|-------------------------------
1. News Feed Generation     | Deque               | O(1) append/pop; maintains order
2. Trending Hashtags        | Counter/Heap        | O(1) count; O(log n) heap ops
3. User Profile Search      | Dictionary/HashMap  | O(1) avg lookup by username
4. Friend Network           | Graph (Adjacency)   | Efficient traversal; represents edges
5. Message Undo Feature     | Stack               | LIFO structure; O(1) push/pop
"""
# ============================================================================
# 1. NEWS FEED (Deque)
# ============================================================================
class NewsFeed:
    """Displays posts chronologically using Deque for O(1) operations."""

    def __init__(self):
        self.feed = deque(maxlen=1000)  # Max 1000 posts

    def add_post(self, post_id: int, username: str, content: str) -> None:
        """Add post to feed (newest first)."""
        self.feed.appendleft({"id": post_id, "user": username, "content": content})

    def get_feed(self, limit: int = 10) -> List[Dict]:
        """Retrieve latest posts efficiently."""
        return list(self.feed)[:limit]
```

```python
# ============================================================================
# 2. TRENDING HASHTAGS (Counter + Heap)
# ============================================================================
class TrendingHashtags:
    """Tracks hashtags using Counter and Heap for O(log n) ranking."""

    def __init__(self):
        self.hashtag_count = Counter()
        self.heap = []

    def add_hashtag(self, hashtag: str) -> None:
        """Add/increment hashtag count."""
        self.hashtag_count[hashtag] += 1

    def get_top_trends(self, k: int = 10) -> List[Tuple[str, int]]:
        """Get top k trending hashtags efficiently."""
        # Use negative counts for max-heap (Python has min-heap)
        self.heap = [(-count, tag) for tag, count in self.hashtag_count.items()]
        heapq.heapify(self.heap)
        return [(tag, -count) for count, tag in heapq.nsmallest(k, self.heap)]
```

```python
# ============================================================================
# 3. USER PROFILE SEARCH (Dictionary/HashMap)
# ============================================================================
class UserProfiles:
    """O(1) username lookup using HashMap."""

    def __init__(self):
        self.users: Dict[str, Dict] = {}

    def register_user(self, username: str, email: str, bio: str = "") -> None:
        """Add new user profile."""
        self.users[username] = {"email": email, "bio": bio, "created": True}

    def search_user(self, username: str) -> Dict | None:
        """Quick O(1) lookup of user profile."""
        return self.users.get(username, None)

    def user_exists(self, username: str) -> bool:
        """Check user existence."""
        return username in self.users
```

```python
# ============================================================================
# 4. FRIEND NETWORK (Graph - Adjacency List)
# ============================================================================
class FriendNetwork:
    """Represents user connections using adjacency list graph."""

    def __init__(self):
        self.graph: Dict[str, Set[str]] = {}

    def add_user(self, username: str) -> None:
        """Add user to network."""
        if username not in self.graph:
            self.graph[username] = set()

    def add_friendship(self, user1: str, user2: str) -> None:
        """Create bidirectional friendship."""
        self.add_user(user1)
        self.add_user(user2)
        self.graph[user1].add(user2)
        self.graph[user2].add(user1)

    def get_friends(self, username: str) -> Set[str]:
        """Get all friends of a user (O(1) retrieval)."""
        return self.graph.get(username, set())

    def find_mutual_friends(self, user1: str, user2: str) -> Set[str]:
        """Find common friends between two users."""
        friends1 = self.get_friends(user1)
        friends2 = self.get_friends(user2)
        return friends1 & friends2
```

```python
# ============================================================================
# 5. MESSAGE UNDO FEATURE (Stack)
# ============================================================================
class MessageStack:
    """Manages message history with undo using Stack (LIFO)."""

    def __init__(self):
        self.message_stack = []

    def send_message(self, sender: str, recipient: str, content: str) -> None:
        """Send message and push to stack."""
        msg = {"from": sender, "to": recipient, "content": content}
        self.message_stack.append(msg)

    def undo_last_message(self) -> Dict | None:
        """Remove and return last sent message in O(1)."""
        return self.message_stack.pop() if self.message_stack else None

    def get_message_history(self) -> List[Dict]:
        """Get all sent messages."""
        return self.message_stack.copy()
```

```python
# ============================================================================
# INTEGRATED DEMO
# ============================================================================
def main():
    """Demonstrate all data structures in action."""

    # Initialize all systems
    feed = NewsFeed()
    trends = TrendingHashtags()
    profiles = UserProfiles()
    network = FriendNetwork()
    messages = MessageStack()

    # Register users
    profiles.register_user("alice", "alice@example.com", "AI enthusiast")
    profiles.register_user("bob", "bob@example.com", "Python developer")

    # Build friend network
    network.add_friendship("alice", "bob")
    network.add_friendship("alice", "charlie")
    network.add_friendship("bob", "charlie")

    # Add posts to feed
    feed.add_post(1, "alice", "Just learned Python! #coding #python")
    feed.add_post(2, "bob", "AI is amazing #AI #ML #coding")

    # Track hashtags
    trends.add_hashtag("#coding")
    trends.add_hashtag("#python")
    trends.add_hashtag("#coding")
    trends.add_hashtag("#AI")
    trends.add_hashtag("#ML")

    # Send messages
    messages.send_message("alice", "bob", "Hey, how are you?")
    messages.send_message("bob", "alice", "Great!")

    # Display results
    print("📰 NEWS FEED:", feed.get_feed(2))
    print("\n🔥 TOP TRENDS:", trends.get_top_trends(3))
    print("\n👤 USER SEARCH:", profiles.search_user("alice"))
    print("\n👥 ALICE'S FRIENDS:", network.get_friends("alice"))
    print("\n💬 LAST MESSAGE UNDO:", messages.undo_last_message())
    print("\n📜 MESSAGE HISTORY:", messages.get_message_history())


if __name__ == "__main__":
    main()
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\14.py"
📰 NEWS FEED: [{'id': 2, 'user': 'bob', 'content': 'AI is amazing #AI #ML #coding'}, {'id': 1, 'user': 'alice', 'content': 'Just learned Python! #coding #python'}]

🔥 TOP TRENDS: [('#coding', 2), ('#AI', 1), ('#ML', 1)]

👤 USER SEARCH: {'email': 'alice@example.com', 'bio': 'AI enthusiast', 'created': True}

👥 ALICE'S FRIENDS: {'charlie', 'bob'}

💬 LAST MESSAGE UNDO: {'from': 'bob', 'to': 'alice', 'content': 'Great!'}

📜 MESSAGE HISTORY: [{'from': 'alice', 'to': 'bob', 'content': 'Hey, how are you?'}]
```

Justification:

- Different data structures were chosen based on system needs, such as graphs for friend networks and stacks for message undo.
- This improves data organization, fast searching, and efficient feature implementation.

Question:15

Task #15: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
from collections import deque, defaultdict
import heapq
from datetime import datetime
from collections import deque

"""
Smart University Hostel Management System
Task #15: Data Structure Selection and Implementation

Feature Mapping Table:
┌──────────────────────────┬──────────────────┬──────────────────────────────────┐
│ Feature                  │ Data Structure   │ Justification                    │
├──────────────────────────┼──────────────────┼──────────────────────────────────┤
│ 1. Room Allotment        │ Hash Table       │ O(1) lookup for room availability │
│                          │                  │ and student mapping; efficient   │
│                          │                  │ allocation tracking.             │
├──────────────────────────┼──────────────────┼──────────────────────────────────┤
│ 2. Medical Emergency     │ Priority Queue   │ Critical emergencies processed   │
│    Handling              │                  │ first; priority-based access     │
│                          │                  │ ensures urgent cases handled.    │
├──────────────────────────┼──────────────────┼──────────────────────────────────┤
│ 3. Student Record Retrieval │ Binary Search  │ Fast O(log n) search by roll     │
│                          │ Tree (BST)       │ number; maintains sorted order;  │
│                          │                  │ supports range queries.          │
├──────────────────────────┼──────────────────┼──────────────────────────────────┤
│ 4. Hostel Block          │ Graph            │ Models connections between blocks; │
│    Connectivity          │                  │ path-finding for shortest route; │
│                          │                  │ adjacency relationships.         │
├──────────────────────────┼──────────────────┼──────────────────────────────────┤
│ 5. Visitor Entry Log     │ Queue (Deque)    │ FIFO processing; chronological   │
│                          │                  │ entry/exit tracking; maintains   │
│                          │                  │ temporal order of visits.        │
└──────────────────────────┴──────────────────┴──────────────────────────────────┘
"""
```

```python
# ============================================================================
# FEATURE 1: ROOM ALLOTMENT - Hash Table Implementation
# ============================================================================

class RoomAllotment:
    """Hash Table-based room allocation system."""

    def __init__(self, total_rooms=100):
        """Initialize room allotment with available rooms."""
        self.rooms = {f"R{i:03d}": {"occupied": False, "student_id": None}
                      for i in range(1, total_rooms + 1)}
        self.student_to_room = {}

    def allocate_room(self, student_id, room_id):
        """Allocate a room to a student in O(1) time."""
        if room_id in self.rooms and not self.rooms[room_id]["occupied"]:
            self.rooms[room_id]["occupied"] = True
            self.rooms[room_id]["student_id"] = student_id
            self.student_to_room[student_id] = room_id
            return True
        return False

    def get_available_rooms(self):
        """Get list of available rooms."""
        return [r for r, info in self.rooms.items() if not info["occupied"]]

    def get_student_room(self, student_id):
        """Retrieve room assigned to student."""
        return self.student_to_room.get(student_id, None)
```

```python
# ==============================================================================
# FEATURE 2: MEDICAL EMERGENCY HANDLING - Priority Queue Implementation
# ==============================================================================

class MedicalEmergencyQueue:
    """Priority Queue for managing medical emergencies by severity."""

    def __init__(self):
        """Initialize priority queue for emergencies."""
        self.pq = []
        self.counter = 0

    def add_emergency(self, patient_id, severity, description):
        """
        Add emergency to priority queue.
        Severity: 1 (Critical), 2 (Urgent), 3 (Regular)
        Lower priority values = processed first
        """
        self.counter += 1
        heapq.heappush(self.pq, (severity, self.counter, {
            'patient_id': patient_id,
            'time': datetime.now(),
            'description': description
        }))

    def process_emergency(self):
        """Process next emergency with highest priority."""
        if self.pq:
            severity, _, emergency = heapq.heappop(self.pq)
            return emergency
        return None

    def pending_emergencies(self):
        """Get count of pending emergencies."""
        return len(self.pq)
```

```python
# ============================================================================
# FEATURE 3: STUDENT RECORD RETRIEVAL - Binary Search Tree Implementation
# ============================================================================

class BSTNode:
    """Node for Binary Search Tree."""

    def __init__(self, roll_no, name, contact):
        self.roll_no = roll_no
        self.name = name
        self.contact = contact
        self.left = None
        self.right = None


class StudentBST:
    """BST for efficient student record retrieval by roll number."""

    def __init__(self):
        self.root = None

    def insert(self, roll_no, name, contact):
        """Insert student record in O(log n) average time."""
        if self.root is None:
            self.root = BSTNode(roll_no, name, contact)
        else:
            self._insert_recursive(self.root, roll_no, name, contact)

    def _insert_recursive(self, node, roll_no, name, contact):
        if roll_no < node.roll_no:
            if node.left is None:
                node.left = BSTNode(roll_no, name, contact)
            else:
                self._insert_recursive(node.left, roll_no, name, contact)
        else:
            if node.right is None:
                node.right = BSTNode(roll_no, name, contact)
            else:
                self._insert_recursive(node.right, roll_no, name, contact)

    def search(self, roll_no):
        """Search student by roll number in O(log n) average time."""
        return self._search_recursive(self.root, roll_no)

    def _search_recursive(self, node, roll_no):
        if node is None:
            return None
        if roll_no == node.roll_no:
            return {'roll_no': node.roll_no, 'name': node.name, 'contact': node.contact}
        elif roll_no < node.roll_no:
            return self._search_recursive(node.left, roll_no)
        else:
            return self._search_recursive(node.right, roll_no)
```

```python
# ==============================================================================
# FEATURE 4: HOSTEL BLOCK CONNECTIVITY - Graph Implementation
# ==============================================================================

class HostelBlockConnectivity:
    """Graph-based hostel block connectivity system."""

    def __init__(self):
        self.graph = defaultdict(list)

    def add_connection(self, block1, block2, distance):
        """Add bidirectional connection between blocks."""
        self.graph[block1].append((block2, distance))
        self.graph[block2].append((block1, distance))

    def find_shortest_path(self, start, end):
        """Find shortest path between blocks using BFS."""

        if start not in self.graph or end not in self.graph:
            return None

        queue = deque([(start, [start])])
        visited = {start}

        while queue:
            node, path = queue.popleft()
            if node == end:
                return path

            for neighbor, _ in self.graph[node]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    queue.append((neighbor, path + [neighbor]))

        return None

    def get_adjacent_blocks(self, block):
        """Get blocks adjacent to given block."""
        return [b for b, _ in self.graph.get(block, [])]
```

```python
# ============================================================================
# FEATURE 5: VISITOR ENTRY LOG - Deque Implementation (WORKING EXAMPLE)
# ============================================================================

class VisitorEntryLog:
    """Deque-based visitor entry/exit log system."""

    def __init__(self):
        """Initialize visitor log."""
        self.log = deque()

    def log_entry(self, visitor_id, visitor_name, room_no, entry_time=None):
        """Log visitor entry."""
        if entry_time is None:
            entry_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        entry = {
            'visitor_id': visitor_id,
            'name': visitor_name,
            'room_no': room_no,
            'entry_time': entry_time,
            'exit_time': None
        }
        self.log.append(entry)
        return entry

    def log_exit(self, exit_time=None):
        """Log visitor exit (most recent entry)."""
        if exit_time is None:
            exit_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        if self.log:
            entry = self.log[-1]
            entry['exit_time'] = exit_time
            return entry
        return None

    def get_current_visitors(self):
        """Get all currently present visitors."""
        return [v for v in self.log if v['exit_time'] is None]

    def get_visitor_history(self):
        """Get chronological visitor history."""
        return list(self.log)
```

```python
# ============================================================================
# DEMONSTRATION & TESTING
# ============================================================================

if __name__ == "__main__":
    print("=" * 70)
    print("SMART UNIVERSITY HOSTEL MANAGEMENT SYSTEM - FEATURE 5: VISITOR LOG")
    print("=" * 70)

    # Initialize visitor log
    visitor_log = VisitorEntryLog()

    # Log visitor entries
    print("\n📝 Logging Visitor Entries:")
    print("-" * 70)
    visitor_log.log_entry("V001", "John Doe", "R101", "2024-01-15 10:30:00")
    visitor_log.log_entry("V002", "Jane Smith", "R205", "2024-01-15 10:45:00")
    visitor_log.log_entry("V003", "Robert Brown", "R310", "2024-01-15 11:00:00")

    print("✓ 3 visitors logged successfully")
    print(f"Current visitors: {len(visitor_log.get_current_visitors())}")

    # Log exit
    print("\n🚪 Logging Visitor Exit:")
    print("-" * 70)
    exit_record = visitor_log.log_exit("2024-01-15 11:15:00")
    if exit_record:
        print(f"✓ {exit_record['name']} exited at {exit_record['exit_time']}")
        print(f"Current visitors: {len(visitor_log.get_current_visitors())}")

    # Display complete history
    print("\n📊 Complete Visitor History:")
    print("-" * 70)
    for idx, visitor in enumerate(visitor_log.get_visitor_history(), 1):
        status = "Exited" if visitor['exit_time'] else "Present"
        print(f"{idx}. {visitor['name']} (Room {visitor['room_no']}) - {status}")

    print("\n" + "=" * 70)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\15.py"
======================================================================
SMART UNIVERSITY HOSTEL MANAGEMENT SYSTEM - FEATURE 5: VISITOR LOG
======================================================================

📝 Logging Visitor Entries:
----------------------------------------------------------------------
✓ 3 visitors logged successfully
Current visitors: 3

🚪 Logging Visitor Exit:
----------------------------------------------------------------------
✓ Robert Brown exited at 2024-01-15 11:15:00
Current visitors: 2

📊 Complete Visitor History:
----------------------------------------------------------------------
1. John Doe (Room R101) - Present
2. Jane Smith (Room R205) - Present
3. Robert Brown (Room R310) - Exited

======================================================================
```

Justification:

- Different data structures were chosen based on system needs, such as graphs for friend networks and stacks for message undo.
- This improves data organization, fast searching, and efficient feature implementation.

Question:16

Task #16: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```
from collections import deque
from datetime import datetime
import heapq
from collections import deque as dq

"""
Smart Electricity Distribution System
Analysis and Implementation of Data Structures for Electricity Board Features
"""

# ---------------------------------------------------------------------------
# FEATURE ANALYSIS TABLE
# ===========================================================================
"""
┌──────────────────────────┬──────────────────┬──────────────────────────┐
│ Feature                  │ Data Structure   │ Justification            │
├──────────────────────────┼──────────────────┼──────────────────────────┤
│ 1. Complaint Handling    │ Queue (FIFO)     │ Complaints should be handled │
│                          │                  │ in order of arrival to ensure│
│                          │                  │ fairness and transparency.   │
├──────────────────────────┼──────────────────┼──────────────────────────┤
│ 2. Fault Priority Handling │ Priority Queue   │ Faults must be prioritized   │
│                          │                  │ by severity; critical faults │
│                          │                  │ need immediate attention.    │
├──────────────────────────┼──────────────────┼──────────────────────────┤
│ 3. Consumer Record Lookup │ Hash Table       │ Fast O(1) average lookup for │
│                          │                  │ consumer data by ID; ideal   │
│                          │                  │ for frequent queries.        │
├──────────────────────────┼──────────────────┼──────────────────────────┤
│ 4. Power Grid Connectivity │ Graph            │ Represents network topology; │
│                          │                  │ nodes are substations/lines; │
│                          │                  │ edges show connections.      │
├──────────────────────────┼──────────────────┼──────────────────────────┤
│ 5. Meter Reading History │ Linked List      │ Sequential insertion of      │
│                          │                  │ time based readings; supports│
│                          │                  │ dynamic growth efficiently.  │
└──────────────────────────┴──────────────────┴──────────────────────────┘
"""
```

```python
# ============================================================================
# FEATURE 1: COMPLAINT HANDLING - Queue Implementation
# ============================================================================

class ComplaintQueue:
    """Queue-based complaint management system for FIFO processing."""

    def __init__(self):
        """Initialize complaint queue."""
        self.queue = deque()

    def register_complaint(self, complaint_id, consumer_id, issue):
        """
        Add complaint to queue with timestamp.
        Time Complexity: O(1)
        """
        complaint = {
            'id': complaint_id,
            'consumer_id': consumer_id,
            'issue': issue,
            'timestamp': datetime.now(),
            'status': 'Pending'
        }
        self.queue.append(complaint)
        print(f"✓ Complaint {complaint_id} registered")

    def process_complaint(self):
        """
        Process complaint in FIFO order.
        Time Complexity: O(1)
        """
        if not self.queue:
            print("No complaints to process")
            return None
        complaint = self.queue.popleft()
        complaint['status'] = 'Processed'
        print(f"Processing Complaint {complaint['id']}: {complaint['issue']}")
        return complaint

    def view_queue(self):
        """Display all pending complaints."""
        if not self.queue:
            print("Queue is empty")
            return
        print(f"\n{'ID':<8} {'Consumer':<12} {'Issue':<20} {'Status':<12}")
        print("-" * 55)
        for c in self.queue:
            print(f"{c['id']:<8} {c['consumer_id']:<12} {c['issue']:<20} {c['status']:<12}")
```

```python
# ============================================================================
# FEATURE 2: FAULT PRIORITY HANDLING - Priority Queue Implementation
# ============================================================================

class FaultPriorityQueue:
    """Priority queue for critical fault handling based on severity."""

    SEVERITY_LEVELS = {'Low': 3, 'Medium': 2, 'Critical': 1}

    def __init__(self):
        """Initialize priority queue for faults."""
        self.heap = []

    def report_fault(self, fault_id, location, severity, description):
        """
        Add fault with priority (lower value = higher priority).
        Time Complexity: O(log n)
        """
        priority = self.SEVERITY_LEVELS.get(severity, 3)
        fault = (priority, fault_id, location, severity, description, datetime.now())
        heapq.heappush(self.heap, fault)
        print(f"✓ Fault {fault_id} ({severity}) reported from {location}")

    def get_urgent_fault(self):
        """
        Extract and resolve most critical fault.
        Time Complexity: O(log n)
        """
        if not self.heap:
            print("No faults to resolve")
            return None
        _, fault_id, location, severity, description, timestamp = heapq.heappop(self.heap)
        print(f"🔧 Resolving CRITICAL Fault {fault_id}: {description} at {location}")
        return {'id': fault_id, 'location': location, 'severity': severity}

    def view_faults(self):
        """Display all pending faults sorted by severity."""
        if not self.heap:
            print("No faults pending")
            return
        print(f"\n{'Priority':<12} {'Fault ID':<10} {'Location':<15} {'Severity':<12} {'Description':<25}")
        print("-" * 75)
        for priority, fid, loc, sev, desc, _ in sorted(self.heap):
            print(f"{sev:<12} {fid:<10} {loc:<15} {sev:<12} {desc:<25}")
```

```python
# ============================================================================
# FEATURE 3: CONSUMER RECORD LOOKUP - Hash Table Implementation
# ============================================================================

class ConsumerRegistry:
    """Hash table for O(1) consumer record lookup by ID."""

    def __init__(self):
        """Initialize consumer registry (hash table)."""
        self.consumers = {}

    def register_consumer(self, consumer_id, name, address, contact, meter_id):
        """
        Add consumer record to registry.
        Time Complexity: O(1) average
        """
        self.consumers[consumer_id] = {
            'name': name,
            'address': address,
            'contact': contact,
            'meter_id': meter_id,
            'balance': 0.0,
            'status': 'Active'
        }
        print(f"✓ Consumer {consumer_id} registered")

    def lookup_consumer(self, consumer_id):
        """
        Fast lookup of consumer by ID.
        Time Complexity: O(1) average
        """
        if consumer_id not in self.consumers:
            print(f"✗ Consumer {consumer_id} not found")
            return None
        return self.consumers[consumer_id]

    def update_balance(self, consumer_id, amount):
        """Update consumer bill balance."""
        if consumer_id in self.consumers:
            self.consumers[consumer_id]['balance'] += amount
            print(f"✓ Balance updated for {consumer_id}: ₹{self.consumers[consumer_id]['balance']:.2f}")

    def view_all_consumers(self):
        """Display all registered consumers."""
        if not self.consumers:
            print("No consumers registered")
            return
        print(f"\n{'ID':<12} {'Name':<15} {'Meter ID':<10} {'Balance':<12} {'Status':<10}")
        print("-" * 60)
        for cid, data in self.consumers.items():
            print(f"{cid:<12} {data['name']:<15} {data['meter_id']:<10} ₹{data['balance']:<11.2f} {data['status']:<10}")
```

```python
# FEATURE 4: POWER GRID CONNECTIVITY - Graph Implementation
# ==========================================================================

class PowerGridGraph:
    """Graph to represent power grid topology and connectivity."""

    def __init__(self):
        """Initialize power grid as adjacency list."""
        self.graph = {}

    def add_substation(self, station_id, capacity):
        """Add substation node to grid."""
        if station_id not in self.graph:
            self.graph[station_id] = {'neighbors': [], 'capacity': capacity}
            print(f"✓ Substation {station_id} added (Capacity: {capacity} MW)")

    def connect_stations(self, station1, station2, line_capacity):
        """
        Connect two substations (undirected edge).
        Time Complexity: O(1)
        """
        if station1 in self.graph and station2 in self.graph:
            self.graph[station1]['neighbors'].append({'to': station2, 'capacity': line_capacity})
            self.graph[station2]['neighbors'].append({'to': station1, 'capacity': line_capacity})
            print(f"✓ Connection: {station1} <--> {station2} ({line_capacity} MW)")

    def find_path_bfs(self, start, end):
        """
        Find path between substations using BFS.
        Time Complexity: O(V + E)
        """
        if start not in self.graph or end not in self.graph:
            return None

        queue = dq([(start, [start])])
        visited = {start}

        while queue:
            node, path = queue.popleft()
            if node == end:
                return path
            for neighbor in self.graph[node]['neighbors']:
                if neighbor['to'] not in visited:
                    visited.add(neighbor['to'])
                    queue.append((neighbor['to'], path + [neighbor['to']]))
        return None

    def view_grid(self):
        """Display power grid topology."""
        print("\n📊 Power Grid Topology:")
        print("-" * 50)
        for station, data in self.graph.items():
            connections = ", ".join([n['to'] for n in data['neighbors']])
            print(f"Station {station} (Cap: {data['capacity']} MW) → Connected to: {connections}")
```

```python
# ============================================================================
# FEATURE 5: METER READING HISTORY - Linked List Implementation
# ============================================================================

class MeterReading:
    """Node for meter reading linked list."""
    def __init__(self, reading_id, reading_value, date):
        self.reading_id = reading_id
        self.reading_value = reading_value
        self.date = date
        self.next = None

class MeterReadingHistory:
    """Linked list for chronological meter reading tracking."""

    def __init__(self, meter_id):
        """Initialize meter reading history."""
        self.meter_id = meter_id
        self.head = None

    def add_reading(self, reading_id, value):
        """
        Add reading to history (prepend for recency).
        Time Complexity: O(1)
        """
        new_reading = MeterReading(reading_id, value, datetime.now())
        new_reading.next = self.head
        self.head = new_reading
        print(f"✓ Reading {reading_id} added: {value} units")

    def calculate_consumption(self, num_readings=2):
        """Calculate consumption between recent readings."""
        if not self.head or not self.head.next:
            print("Insufficient readings for comparison")
            return None

        current = self.head
        for _ in range(num_readings - 1):
            if current.next is None:
                break
            current = current.next

        consumption = self.head.reading_value - current.reading_value
        print(f"✓ Consumption ({num_readings} readings): {consumption} units")
        return consumption

    def view_history(self, limit=10):
        """
        Display reading history.
        Time Complexity: O(n)
        """
        if not self.head:
            print("No readings available")
            return

        print(f"\n{'Reading ID':<15} {'Value (units)':<15} {'Date':<25}")
        print("-" * 55)
        current = self.head
        count = 0
        while current and count < limit:
            print(f"{current.reading_id:<15} {current.reading_value:<15} {current.date.strftime('%Y-%m-%d %H:%M:%S'):<25}")
            current = current.next
            count += 1
```

```python
# ============================================================================
# DEMONSTRATION & TESTING
# ============================================================================

def main():
    print("╔" + "═" * 60 + "╗")
    print("│" + " SMART ELECTRICITY DISTRIBUTION SYSTEM ".center(60) + "│")
    print("╚" + "═" * 60 + "╝\n")

    # ---- FEATURE 1: Complaint Handling ----
    print("\n" + "-" * 65)
    print("FEATURE 1: COMPLAINT HANDLING (QUEUE)")
    print("-" * 65)
    complaint_system = ComplaintQueue()
    complaint_system.register_complaint("C001", "CONS123", "Power outage")
    complaint_system.register_complaint("C002", "CONS456", "High bill")
    complaint_system.register_complaint("C003", "CONS789", "Meter malfunction")
    complaint_system.view_queue()
    complaint_system.process_complaint()
    print()

    # ---- FEATURE 2: Fault Priority Handling ----
    print("\n" + "-" * 65)
    print("FEATURE 2: FAULT PRIORITY HANDLING (PRIORITY QUEUE)")
    print("-" * 65)
    fault_system = FaultPriorityQueue()
    fault_system.report_fault("F001", "Substation A", "Low", "Loose connection")
    fault_system.report_fault("F002", "Substation B", "Critical", "Transformer failure")
    fault_system.report_fault("F003", "Line C", "Medium", "Wire damage")
    fault_system.view_faults()
    fault_system.get_urgent_fault()
    print()

    # ---- FEATURE 3: Consumer Record Lookup ----
    print("\n" + "-" * 65)
    print("FEATURE 3: CONSUMER RECORD LOOKUP (HASH TABLE)")
    print("-" * 65)
    registry = ConsumerRegistry()
    registry.register_consumer("CONS123", "Rajesh Kumar", "123 Main St", "9876543210", "M001")
    registry.register_consumer("CONS456", "Priya Singh", "456 Oak Ave", "9876543211", "M002")
    registry.register_consumer("CONS789", "Amit Patel", "789 Pine Rd", "9876543212", "M003")
    registry.update_balance("CONS123", 2500)
    registry.update_balance("CONS456", 1800)
    consumer = registry.lookup_consumer("CONS123")
    registry.view_all_consumers()
    print()

    # ---- FEATURE 4: Power Grid Connectivity ----
    print("\n" + "-" * 65)
    print("FEATURE 4: POWER GRID CONNECTIVITY (GRAPH)")
    print("-" * 65)
    grid = PowerGridGraph()
    grid.add_substation("Sub_A", 500)
    grid.add_substation("Sub_B", 400)
    grid.add_substation("Sub_C", 300)
    grid.add_substation("Sub_D", 350)
    grid.connect_stations("Sub_A", "Sub_B", 200)
    grid.connect_stations("Sub_B", "Sub_C", 150)
    grid.connect_stations("Sub_C", "Sub_D", 180)
    path = grid.find_path_bfs("Sub_A", "Sub_D")
    print(f"✓ Power Path A→D: {' → '.join(path)}" if path else "No path found")
    grid.view_grid()
    print()

    # ---- FEATURE 5: Meter Reading History ----
    print("\n" + "-" * 65)
    print("FEATURE 5: METER READING HISTORY (LINKED LIST)")
    print("-" * 65)
    meter = MeterReadingHistory("M001")
    meter.add_reading("R001", 1000)
    meter.add_reading("R002", 1250)
    meter.add_reading("R003", 1500)
    meter.calculate_consumption(2)
    meter.view_history()

    print("\n" + "╔" + "═" * 60 + "╗")
    print("│" + " SYSTEM READY FOR DEPLOYMENT ".center(60) + "│")
    print("╚" + "═" * 60 + "╝")

if __name__ == "__main__":
    main()
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\16.py"
╔═══════════════════════════════════════════════════════════╗
║           SMART ELECTRICITY DISTRIBUTION SYSTEM           ╒
╚═══════════════════════════════════════════════════════════╝


────────────────────────────────────────────────────────────
FEATURE 1: COMPLAINT HANDLING (QUEUE)
────────────────────────────────────────────────────────────
✓ Complaint C001 registered
✓ Complaint C002 registered
✓ Complaint C003 registered

ID        Consumer      Issue                  Status
--------------------------------------------------
C001      CONS123       Power outage           Pending
C002      CONS456       High bill              Pending
C003      CONS789       Meter malfunction      Pending
Processing Complaint C001: Power outage


────────────────────────────────────────────────────────────
FEATURE 2: FAULT PRIORITY HANDLING (PRIORITY QUEUE)
────────────────────────────────────────────────────────────
✓ Fault F001 (Low) reported from Substation A
✓ Fault F002 (Critical) reported from Substation B
✓ Fault F003 (Medium) reported from Line C

Priority      Fault ID    Location         Severity      Description
------------------------------------------------------------------------
Critical      F002        Substation B     Critical      Transformer failure
Medium        F003        Line C           Medium        Wire damage
Low           F001        Substation A     Low           Loose connection
🔧 Resolving CRITICAL Fault F002: Transformer failure at Substation B


────────────────────────────────────────────────────────────
FEATURE 3: CONSUMER RECORD LOOKUP (HASH TABLE)
────────────────────────────────────────────────────────────
✓ Consumer CONS123 registered
✓ Consumer CONS456 registered
✓ Consumer CONS789 registered
✓ Balance updated for CONS123: ₹2500.00
✓ Balance updated for CONS456: ₹1800.00

ID            Name             Meter ID    Balance       Status
-------------------------------------------------------------
CONS123       Rajesh Kumar     M001        ₹2500.00      Active
CONS456       Priya Singh      M002        ₹1800.00      Active
CONS789       Amit Patel       M003        ₹0.00         Active
```

```
────────────────────────────────────────────────────────────
FEATURE 4: POWER GRID CONNECTIVITY (GRAPH)
────────────────────────────────────────────────────────────
✓ Substation Sub_A added (Capacity: 500 MW)
✓ Substation Sub_B added (Capacity: 400 MW)
✓ Substation Sub_C added (Capacity: 300 MW)
✓ Substation Sub_D added (Capacity: 350 MW)
✓ Connection: Sub_A <--> Sub_B (200 MW)
✓ Connection: Sub_B <--> Sub_C (150 MW)
✓ Connection: Sub_C <--> Sub_D (180 MW)
✓ Power Path A→D: Sub_A → Sub_B → Sub_C → Sub_D

📊 Power Grid Topology:
--------------------------------------------------
Station Sub_A (Cap: 500 MW) → Connected to: Sub_B
Station Sub_B (Cap: 400 MW) → Connected to: Sub_A, Sub_C
Station Sub_C (Cap: 300 MW) → Connected to: Sub_B, Sub_D
Station Sub_D (Cap: 350 MW) → Connected to: Sub_C


────────────────────────────────────────────────────────────
FEATURE 5: METER READING HISTORY (LINKED LIST)
────────────────────────────────────────────────────────────
✓ Reading R001 added: 1000 units
✓ Reading R002 added: 1250 units
✓ Reading R003 added: 1500 units
✓ Consumption (2 readings): 250 units

Reading ID       Value (units)    Date
-------------------------------------------------
R003             1500             2026-03-11 21:12:50
R002             1250             2026-03-11 21:12:50
R001             1000             2026-03-11 21:12:50

╔═══════════════════════════════════════════════════════════╗
║                SYSTEM READY FOR DEPLOYMENT                ╒
╚═══════════════════════════════════════════════════════════╝
```

Justification:

- The logic was organized into separate functions, which improves modularity and readability.
- This makes the code easier to maintain and reuse for similar problems.

Question:17

Task #17: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
import heapq
from collections import deque, defaultdict
from dataclasses import dataclass
from typing import Optional

"""
Smart Logistics & Warehouse Management System
Task #17: Data Structure Selection & Implementation
"""

# ============================================================================
# FEATURE ANALYSIS & DATA STRUCTURE MAPPING
# ============================================================================
"""
FEATURE → DATA STRUCTURE → JUSTIFICATION TABLE
────────────────────────────────────────────────────────────────────────────

1. SHIPMENT PROCESSING
    Data Structure: Queue
    Justification: Shipments must be processed in FIFO order. Packages arrive
    sequentially and should be handled in the order they arrive to maintain
    fairness and prevent bottlenecks.

2. URGENT SHIPMENT PRIORITY
    Data Structure: Priority Queue
    Justification: Urgent shipments need to bypass regular queues based on
    priority levels. A min/max heap efficiently retrieves the highest-priority
    item in O(log n) time.

3. PRODUCT LOOKUP BY CODE
    Data Structure: Hash Table (Dictionary)
    Justification: Fast O(1) average-case lookup time for product details.
    Hash tables excel at key-value retrieval without needing to traverse data.

4. WAREHOUSE NETWORK CONNECTIVITY
    Data Structure: Graph
    Justification: Warehouses form nodes with distribution routes as edges.
    Graphs efficiently represent network topology and enable pathfinding
    algorithms like Dijkstra's for optimal routing.

5. INVENTORY ADD/REMOVE OPERATIONS
    Data Structure: Deque (Double-ended Queue)
    Justification: Inventory can be added/removed from both ends efficiently.
    Deques support O(1) operations at both ends, ideal for LIFO/FIFO flexibility.

────────────────────────────────────────────────────────────────────────────
"""
```

```python
# ============================================================================
# FEATURE 1: SHIPMENT PROCESSING (Queue)
# ============================================================================

class ShipmentQueue:
    """
    FIFO queue for processing shipments in arrival order.
    Ensures fair and sequential handling of all packages.
    """

    def __init__(self):
        self.queue = deque()

    def add_shipment(self, shipment_id: str, destination: str):
        """Add a shipment to the queue."""
        self.queue.append({"id": shipment_id, "destination": destination})

    def process_shipment(self) -> Optional[dict]:
        """Process and remove the oldest shipment."""
        return self.queue.popleft() if self.queue else None

    def display_queue(self):
        """Display all pending shipments."""
        return list(self.queue)


# ============================================================================
# FEATURE 2: URGENT SHIPMENT PRIORITY (Priority Queue)
# ============================================================================

@dataclass
class UrgentShipment:
    """Data class for urgent shipment with priority."""
    priority: int  # 1=Highest, 5=Lowest
    shipment_id: str
    destination: str

    def __lt__(self, other):
        return self.priority < other.priority


class PriorityShipmentManager:
    """
    Priority queue for handling urgent shipments.
    Higher priority shipments are processed first.
    """

    def __init__(self):
        self.heap = []
        self.counter = 0

    def add_urgent_shipment(self, shipment_id: str, destination: str, priority: int):
        """Add shipment with priority (1=highest, 5=lowest)."""
        self.counter += 1
        shipment = UrgentShipment(priority, shipment_id, destination)
        heapq.heappush(self.heap, (priority, self.counter, shipment))

    def process_urgent_shipment(self) -> Optional[UrgentShipment]:
        """Extract and return highest priority shipment."""
        return heapq.heappop(self.heap)[2] if self.heap else None

    def display_priorities(self):
        """Display shipments in priority order."""
        return sorted(self.heap)
```

```python
# ============================================================================
# FEATURE 3: PRODUCT LOOKUP BY CODE (Hash Table)
# ============================================================================

class ProductCatalog:
    """
    Hash table-based product lookup system.
    O(1) average access time for product details.
    """

    def __init__(self):
        self.catalog = {}

    def add_product(self, code: str, name: str, price: float, quantity: int):
        """Add product to catalog using unique code as key."""
        self.catalog[code] = {
            "name": name,
            "price": price,
            "quantity": quantity
        }

    def lookup_product(self, code: str) -> Optional[dict]:
        """O(1) lookup product by code."""
        return self.catalog.get(code, None)

    def remove_product(self, code: str) -> bool:
        """Remove product from catalog."""
        if code in self.catalog:
            del self.catalog[code]
            return True
        return False

    def display_catalog(self):
        """Display all products."""
        return self.catalog


# ============================================================================
# FEATURE 4: WAREHOUSE NETWORK CONNECTIVITY (Graph)
# ============================================================================

class WarehouseNetwork:
    """
    Graph representation of warehouse network.
    Enables optimal routing and connectivity analysis.
    """

    def __init__(self):
        self.graph = defaultdict(list)

    def add_route(self, warehouse_a: str, warehouse_b: str, distance: int):
        """Add bidirectional route between warehouses."""
        self.graph[warehouse_a].append((warehouse_b, distance))
        self.graph[warehouse_b].append((warehouse_a, distance))

    def find_shortest_path(self, start: str, end: str) -> list:
        """BFS to find shortest path between warehouses."""
        visited = set()
        queue = deque([(start, [start])])

        while queue:
            node, path = queue.popleft()
            if node == end:
                return path

            if node not in visited:
                visited.add(node)
                for neighbor, _ in self.graph[node]:
                    queue.append((neighbor, path + [neighbor]))

        return []

    def display_network(self):
        """Display warehouse connections."""
        return dict(self.graph)
```

```python
# ============================================================================
# FEATURE 5: INVENTORY ADD/REMOVE OPERATIONS (Deque)
# ============================================================================

class InventoryManager:
    """
    Deque-based inventory system.
    Supports efficient add/remove from both ends.
    """

    def __init__(self):
        self.inventory = deque()

    def add_to_front(self, item_id: str, quantity: int):
        """Add items to front (priority stock)."""
        self.inventory.appendleft({"id": item_id, "qty": quantity})

    def add_to_back(self, item_id: str, quantity: int):
        """Add items to back (standard stock)."""
        self.inventory.append({"id": item_id, "qty": quantity})

    def remove_from_front(self) -> Optional[dict]:
        """Remove items from front (LIFO for urgent access)."""
        return self.inventory.popleft() if self.inventory else None

    def remove_from_back(self) -> Optional[dict]:
        """Remove items from back (FIFO for regular access)."""
        return self.inventory.pop() if self.inventory else None

    def display_inventory(self):
        """Display all inventory items."""
        return list(self.inventory)
```

```python
# ============================================================================
# DEMONSTRATION & TESTING
# ============================================================================

if __name__ == "__main__":
    print("=" * 70)
    print("SMART LOGISTICS & WAREHOUSE MANAGEMENT SYSTEM")
    print("=" * 70)

    # Feature 1: Shipment Processing
    print("\n📦 1  SHIPMENT PROCESSING (Queue)")
    print("-" * 70)
    sq = ShipmentQueue()
    sq.add_shipment("SHIP001", "New York")
    sq.add_shipment("SHIP002", "Los Angeles")
    sq.add_shipment("SHIP003", "Chicago")
    print(f"Pending shipments: {sq.display_queue()}")
    processed = sq.process_shipment()
    print(f"Processed: {processed}")

    # Feature 2: Urgent Shipment Priority
    print("\n🚨 2  URGENT SHIPMENT PRIORITY (Priority Queue)")
    print("-" * 70)
    pq = PriorityShipmentManager()
    pq.add_urgent_shipment("URGENT001", "Miami", 3)
    pq.add_urgent_shipment("URGENT002", "Boston", 1)
    pq.add_urgent_shipment("URGENT003", "Denver", 2)
    urgent = pq.process_urgent_shipment()
    print(f"Highest priority shipment: {urgent}")

    # Feature 3: Product Lookup
    print("\n🔍 3  PRODUCT LOOKUP BY CODE (Hash Table)")
    print("-" * 70)
    catalog = ProductCatalog()
    catalog.add_product("PROD001", "Laptop", 999.99, 50)
    catalog.add_product("PROD002", "Mouse", 29.99, 200)
    catalog.add_product("PROD003", "Keyboard", 79.99, 150)
    print(f"Lookup PROD002: {catalog.lookup_product('PROD002')}")
    print(f"Catalog size: {len(catalog.display_catalog())}")

    # Feature 4: Warehouse Network
    print("\n🗺 4  WAREHOUSE NETWORK CONNECTIVITY (Graph)")
    print("-" * 70)
    network = WarehouseNetwork()
    network.add_route("WH_A", "WH_B", 50)
    network.add_route("WH_B", "WH_C", 30)
    network.add_route("WH_C", "WH_D", 40)
    network.add_route("WH_A", "WH_D", 100)
    path = network.find_shortest_path("WH_A", "WH_D")
    print(f"Shortest path from WH_A to WH_D: {path}")

    # Feature 5: Inventory Management
    print("\n📦 5  INVENTORY ADD/REMOVE OPERATIONS (Deque)")
    print("-" * 70)
    inv = InventoryManager()
    inv.add_to_back("ITEM001", 100)
    inv.add_to_back("ITEM002", 50)
    inv.add_to_front("URGENT_ITEM", 10)
    print(f"Current inventory: {inv.display_inventory()}")
    removed = inv.remove_from_front()
    print(f"Removed from front: {removed}")
    print(f"Remaining inventory: {inv.display_inventory()}")

    print("\n" + "=" * 70)
    print("✅ All features implemented successfully!")
    print("=" * 70)
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\17.py"
======================================================================
SMART LOGISTICS & WAREHOUSE MANAGEMENT SYSTEM
======================================================================

1️⃣  SHIPMENT PROCESSING (Queue)
----------------------------------------------------------------------
Pending shipments: [{'id': 'SHIP001', 'destination': 'New York'}, {'id': 'SHIP002', 'destination': 'Los Angeles'}, {'id': 'SHIP003',
'destination': 'Chicago'}]
Processed: {'id': 'SHIP001', 'destination': 'New York'}

2️⃣  URGENT SHIPMENT PRIORITY (Priority Queue)
----------------------------------------------------------------------
Highest priority shipment: UrgentShipment(priority=1, shipment_id='URGENT002', destination='Boston')

3️⃣  PRODUCT LOOKUP BY CODE (Hash Table)
----------------------------------------------------------------------
Lookup PROD002: {'name': 'Mouse', 'price': 29.99, 'quantity': 200}
Catalog size: 3

4️⃣  WAREHOUSE NETWORK CONNECTIVITY (Graph)
----------------------------------------------------------------------
Shortest path from WH_A to WH_D: ['WH_A', 'WH_D']

5️⃣  INVENTORY ADD/REMOVE OPERATIONS (Deque)
----------------------------------------------------------------------
Current inventory: [{'id': 'URGENT_ITEM', 'qty': 10}, {'id': 'ITEM001', 'qty': 100}, {'id': 'ITEM002', 'qty': 50}]
Removed from front: {'id': 'URGENT_ITEM', 'qty': 10}
Remaining inventory: [{'id': 'ITEM001', 'qty': 100}, {'id': 'ITEM002', 'qty': 50}]

======================================================================
✅ All features implemented successfully!
======================================================================
```

Justification:

- The optimized approach reduces unnecessary computations, improving program efficiency.
- Using a structured method makes the code clearer and easier to debug.

Question:18

Task 18 (Collatz Sequence Generator – Test Case Design)

• Function: Generate Collatz sequence until reaching 1.
• Test Cases to Design:
• Normal: 6 → [6,3,10,5,16,8,4,2,1]
• Edge: 1 → [1]
• Negative: -5
• Large: 27 (well-known long sequence)
• Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:
• Takes an integer n as input.
• Generates the Collatz sequence (also called the 3n+1 sequence).
• The rules are:
o If n is even → next = n / 2.
o If n is odd → next = 3n + 1.
• Repeat until we reach 1.
• Return the full sequence as a list.

Example

Input: 6

Steps:
• 6 (even → 6/2 = 3)
• 3 (odd → 3*3+1 = 10)
• 10 (even → 10/2 = 5)

- 5 (odd → 3*5+1 = 16)
- 16 (even → 16/2 = 8)
- 8 (even → 8/2 = 4)
- 4 (even → 4/2 = 2)
- 2 (even → 2/2 = 1)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

Code:

```python
import pytest
def collatz_sequence(n):
    """
    Generate the Collatz sequence until reaching 1.

    Args:
        n: A positive integer

    Returns:
        List containing the Collatz sequence

    Raises:
        ValueError: If n is not a positive integer
    """
    if not isinstance(n, int) or n < 1:
        raise ValueError("Input must be a positive integer")
    sequence = []
    while n != 1:
        sequence.append(n)
        n = n // 2 if n % 2 == 0 else 3 * n + 1
    sequence.append(1)
    return sequence
# Test cases using pytest
def test_normal_case():
    """Test normal case: 6"""
    assert collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]
def test_edge_case():
    """Test edge case: 1"""
    assert collatz_sequence(1) == [1]
def test_large_case():
    """Test large case: 27 (long sequence)"""
    result = collatz_sequence(27)
    assert result[0] == 27
    assert result[-1] == 1
    assert len(result) == 112
def test_power_of_two():
    """Test power of 2: 16"""
    assert collatz_sequence(16) == [16, 8, 4, 2, 1]
def test_negative_input():
    """Test negative input: -5"""
    with pytest.raises(ValueError):
        collatz_sequence(-5)
def test_zero_input():
    """Test zero input"""
    with pytest.raises(ValueError):
        collatz_sequence(0)
def test_non_integer_input():
    """Test non-integer input"""
    with pytest.raises(ValueError):
        collatz_sequence(3.5)
if __name__ == "__main__":
    try:
        number = int(input("Enter a positive integer: "))
        sequence = collatz_sequence(number)
        print("Collatz sequence:", sequence)
    except ValueError as error:
        print(f"Error: {error}")
```

Output:

```
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\18.py"
Enter a positive integer: 12
Collatz sequence: [12, 6, 3, 10, 5, 16, 8, 4, 2, 1]
```

Justification:

- The problem was solved using an appropriate data structure, which ensures faster data access.
- This improves performance and scalability when handling larger inputs.

Question:19

Task 19 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.
(Starts with 2,1, then Fn = Fn-1 + Fn-2)
• Test Cases to Design:
• Normal: 5 → [2, 1, 3, 4, 7]
• Edge: 1 → [2]
• Negative: -5 → Error
• Large: 10 (last element = 76).
• Requirement: Validate correctness with pytest.

Code:

```python
import pytest
def lucas_sequence(n):
    if n < 1:
        raise ValueError("n must be a positive integer")
    if n == 1:
        return [2]
    lucas = [2, 1]
    for _ in range(n - 2):
        lucas.append(lucas[-1] + lucas[-2])
    return lucas[:n]
# Test cases using pytest
def test_lucas_normal():
    """Test normal case with 5 terms"""
    assert lucas_sequence(5) == [2, 1, 3, 4, 7]
def test_lucas_edge():
    """Test edge case with 1 term"""
    assert lucas_sequence(1) == [2]
def test_lucas_negative():
    """Test negative input"""
    with pytest.raises(ValueError):
        lucas_sequence(-5)
def test_lucas_zero():
    """Test zero input"""
    with pytest.raises(ValueError):
        lucas_sequence(0)
def test_lucas_large():
    """Test large case with 10 terms"""
    result = lucas_sequence(10)
    assert result[-1] == 76
    assert len(result) == 10
if __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Output:

```
PS C:\Users\PC\OneDrive\Desktop\AIAC> python "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\19.py"
============================================================================= test session starts =============================================================================
platform win32 -- Python 3.15.0a3, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\PC\AppData\Local\Programs\Python\Python315\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\PC\OneDrive\Desktop\AIAC
collected 5 items

24/2/2026/Refactoring/19.py::test_lucas_normal PASSED                                                                                                                    [ 20%]
24/2/2026/Refactoring/19.py::test_lucas_edge PASSED                                                                                                                      [ 40%]
24/2/2026/Refactoring/19.py::test_lucas_negative PASSED                                                                                                                  [ 60%]
24/2/2026/Refactoring/19.py::test_lucas_zero PASSED                                                                                                                      [ 80%]
24/2/2026/Refactoring/19.py::test_lucas_large PASSED                                                                                                                     [100%]

============================================================================== 5 passed in 0.04s ==============================================================================
```

Justification:

- Breaking the solution into logical steps/functions improves code organization.
- It also simplifies testing and future modifications.

Question:20

Task 20 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

• Normal: "hello" → (2,3)

• Edge: "" → (0,0)

• Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.

Code:

```python
import unittest


def count_vowels_consonants(text):
    """Return (vowel_count, consonant_count) for alphabetic characters in text."""
    vowels = set("aeiou")
    vowel_count = 0
    consonant_count = 0

    for char in text.lower():
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count

class TestVowelConsonantCounter(unittest.TestCase):
    def test_normal_case(self):
        self.assertEqual(count_vowels_consonants("hello"), (2, 3))

    def test_empty_string(self):
        self.assertEqual(count_vowels_consonants(""), (0, 0))

    def test_only_vowels(self):
        self.assertEqual(count_vowels_consonants("aeiou"), (5, 0))

    def test_only_consonants(self):
        self.assertEqual(count_vowels_consonants("bcdfg"), (0, 5))

    def test_mixed_case(self):
        self.assertEqual(count_vowels_consonants("HeLLo"), (2, 3))

    def test_with_spaces(self):
        self.assertEqual(count_vowels_consonants("hello world"), (3, 7))

    def test_with_special_characters(self):
        self.assertEqual(count_vowels_consonants("hello!@#"), (2, 3))

    def test_long_text(self):
        long_text = "a" * 100 + "b" * 100
        self.assertEqual(count_vowels_consonants(long_text), (100, 100))

    def test_all_vowels_uppercase(self):
        self.assertEqual(count_vowels_consonants("AEIOU"), (5, 0))

    def test_with_numbers(self):
        self.assertEqual(count_vowels_consonants("hello123"), (2, 3))

if __name__ == '__main__':
    unittest.main()
```

Output:

```
[Running] python -u "c:\Users\PC\OneDrive\Desktop\AIAC\24\2\2026\Refactoring\20.py"
..........
----------------------------------------------------------------------
Ran 10 tests in 0.000s

OK
```

Justification:

- The final design follows clean coding principles, making the program readable and structured.
- Proper abstraction and modular design improve maintainability and reusability.